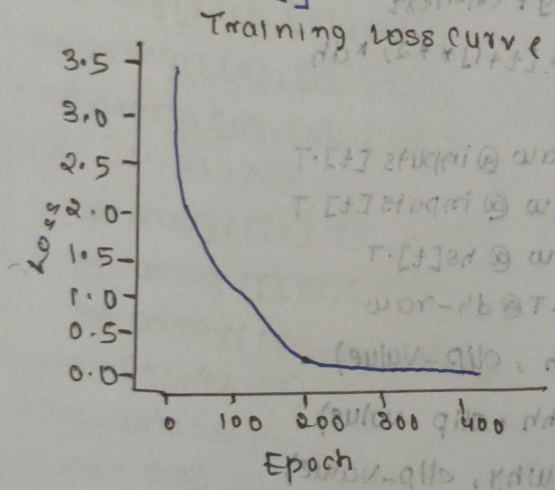
Aim:- Forward pass and backpropagation with random weights

```
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(suppress=True)
def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x / np.sum(exp_x)
def cross_entropy(y_true, y_pred):
    return -np.sum(y_true * np.log(y_pred + 1e-12))
Wxh = np.array([[0.1, 0.2, 0.3], [0.4, 0.5, 0.6]])
Whh = np.array([[0.7, 0.8], [0.9, 1.0]])
Why = np.array([[0.1, 0.2], [0.3, 0.4], [0.5, 0.6]])
bh = np.array([[0.1], [0.2]])
by = np.array([[0.1], [0.2], [0.3]])
x1 = np.array([[1], [0], [0]])
x2 = np.array([[0], [1], [0]])
x3 = np.array([[0], [0], [1]])
inputs = [x1, x2, x3]
t1 = np.array([[0], [1], [0]])
t2 = np.array([[0], [0], [1]])
t3 = np.array([[1], [0], [0]])
targets = [t1, t2, t3]
learning_rate = 0.1
threshold = 0.05
max_epochs = 1000
clip_value = 5
def clip_gradients(grad, clip_value):
    return np.clip(grad, -clip_value, clip_value)
loss_history = []
for epoch in range(max_epochs):
    h0 = np.zeros((2,1))
    hs = [h0]
    ps = []
    Loss = 0
    for t in range(3):
        ht = np.tanh(Wxh @ inputs[t] + Whh @ hs[-1] + bh)
        y = Why @ ht + by
        p = softmax(y)
```

```
        hs.append(ht)
        ps.append(p)
        loss += cross_entropy(targets[t], p)
    loss_history.append(loss)
    if loss < threshold:
        print("Training converged at Epoch:", epoch+1)
        break
    dWxh = np.zeros_like(Wxh)
    dWhh = np.zeros_like(Whh)
    dWhy = np.zeros_like(Why)
    dbh = np.zeros_like(bh)
    dby = np.zeros_like(by)
    dh_next = np.zeros((2,1))
    for t in reversed(range(3)):
        dy = ps[t] - targets[t]
        dWhy += dy @ hs[t+1].T
        dby += dy
        dh = Why.T @ dy + dh_next
        dh_raw = (1 - hs[t+1]**2) * dh
        dbh += dh_raw
        dWxh += dh_raw @ inputs[t].T
        dWhh += dh_raw @ inputs[t].T
        dWhh += dh_raw @ hs[t].T
        dh_next = Whh.T @ dh_raw
    dWxh = clip_gradients(dWxh, clip_value)
    dWhh = clip_gradients(dWhh, clip_value)
    dWhy = clip_gradients(dWhy, clip_value)
    dbh = clip_gradients(dbh, clip_value)
    dby = clip_gradients(dby, clip_value)
    Wxh -= learning_rate * dWxh
    Whh -= learning_rate * dWhh
    Why -= Learning_rate * dWhy
    bh -= Learning_rate * bh
    by -= Learning_rate * dby
    if (epoch+1) % 100 == 0:
        print("Epoch:", epoch+1, "Loss:", Loss)
print("\nFinal Loss:", Loss)
print("\nFinal Predictions:")
for t in range(3):
    print(np.round(ps[t], 1))
```

```
plt.plot(loss_history, color='blue')
plt.title("Training Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

Output:-

```
Epoch: 100  Loss: 0.936083258220132
Epoch: 200  Loss: 0.27059480298641 28
Epoch: 300  Loss: 0.10184475802385928
Epoch: 400  Loss: 0.05550842420245878
Training converged at epoch: 423
Final Loss: 0.49874655771221556
Final predictions:
[[0.] [1.] [0.]]
[[0.] [0.] [1.]]
[[1.] [0.] [0.]]
```